# 1. Semantic Understanding and Command Following (Voice Version)

## 1. Course Content

1. Learn the complete workflow of natural interaction with an embodied intelligent robot through voice dialogue;
2. Master the complete closed loop in which the speech recognition large model (ASR) converts speech into text, the text generation large model parses semantic actions, and speech synthesis (TTS) together with low-level action services drives the car to execute the specified actions;
3. Verify the large model's instruction-following capability for chassis motion displacement, speed adjustment ("move forward quickly", "retreat slowly like a turtle"), steering angle solving, and anthropomorphic entertainment performances (dance + joke telling);
4. Analyze the underlying core source code and common microphone troubleshooting solutions.

---

## 2. Preparation

### 2.1 Description

> This lesson is applicable to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You need to first enter the Docker container before executing ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Directly enter the commands in the vehicle terminal; there is no need to enter Docker.

### 2.2 Start the Dify Local Large Model Environment

Open the vehicle host terminal and run the script to start the Dify service:

```
sh ~/bringup_dify.sh
```




---

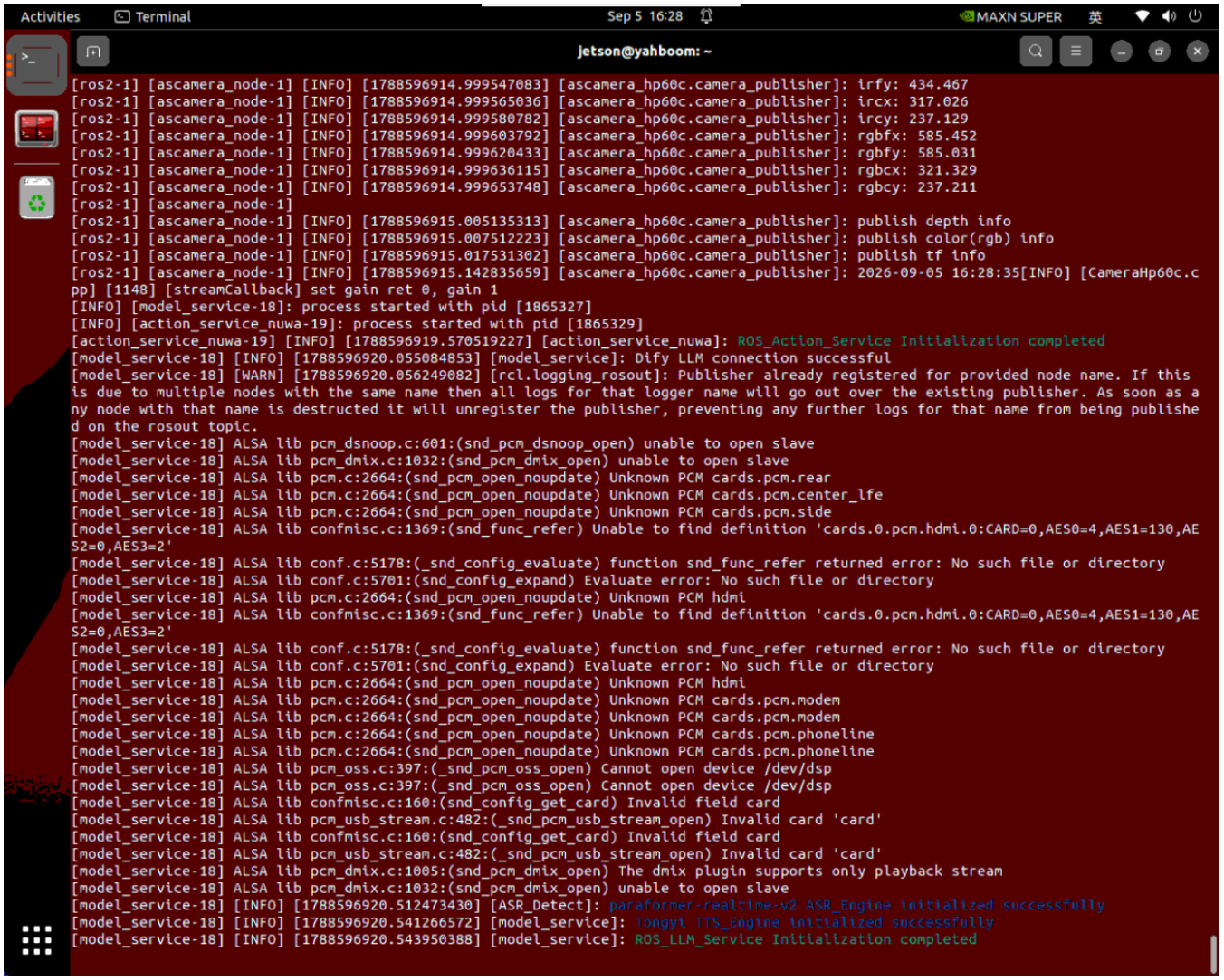
# 3. Running the Cases

## 3.1 Start the Embodied Intelligence Master Control Node (Voice Mode)

Open a terminal on the vehicle and enter the following startup command (the default is microphone voice interaction mode):

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After the node initialization is complete, the terminal displays a prompt indicating that all sensors and models have been loaded successfully:

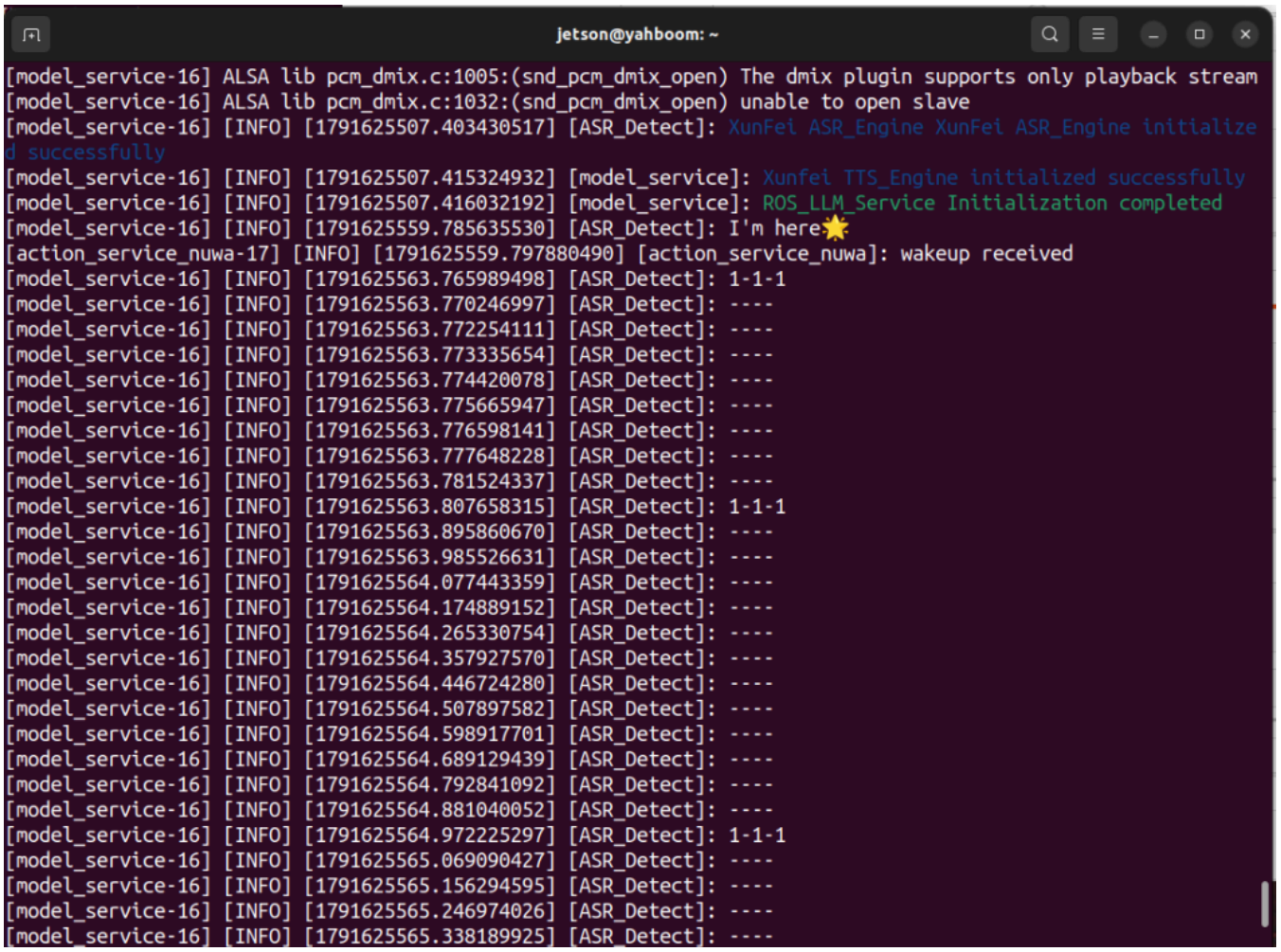



## 3.2 Classic Test Cases

### Interaction Specification:

1. First clearly speak the wake word: **"Hi,yahboom"** to wake up the robot;
2. Listen for the car's voice response: **"I'm here, please give me instructions"** or various personalized wake-up voices;

3. After the response ends, you can speak the test instruction to the robot. (When sound is active, `1` is printed; when there is no sound, `-` is printed. If the trailing silence exceeds 450 ms, the audio is automatically truncated and uploaded for recognition):

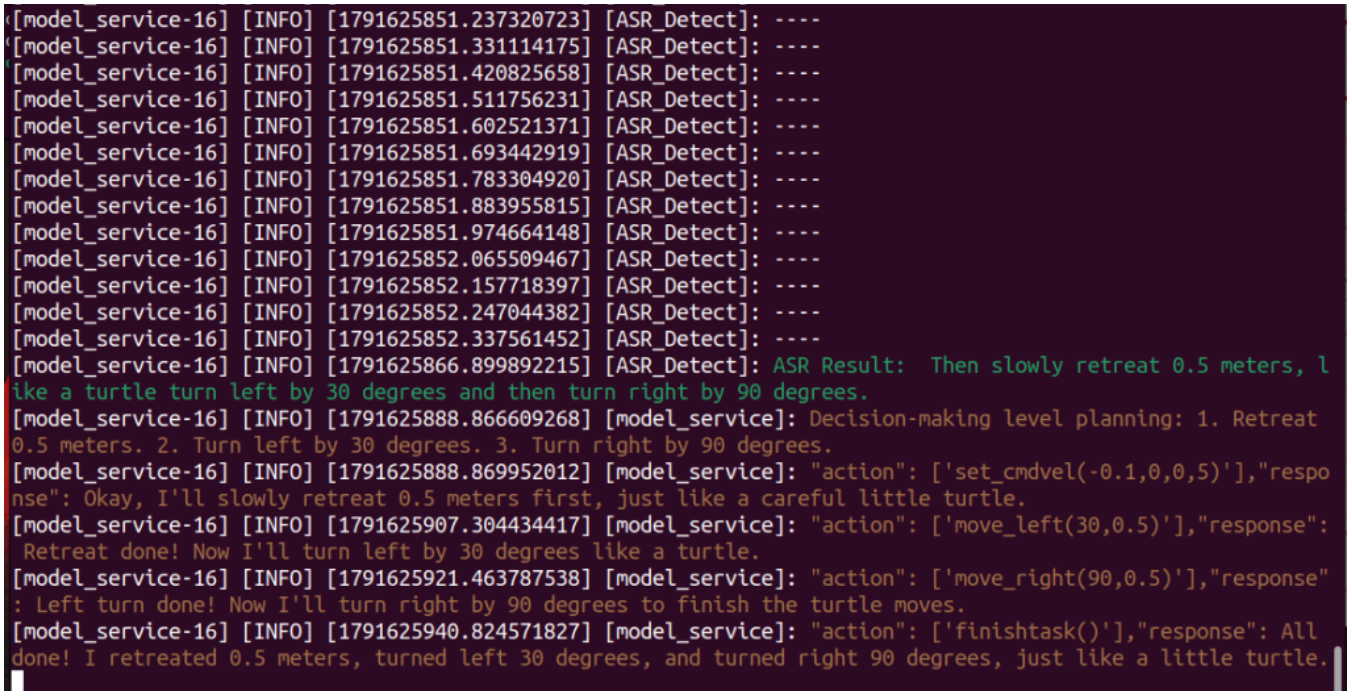



## Case 1: Chassis Motion Speed and Composite Displacement Control

After waking up yahboom, speak the instruction to the car: **(Note: left and right turns require a large amount of space. It is recommended to raise the car for testing or ensure there is sufficient space around the car)**

```
First, move forward quickly for 1 meter, then slowly retreat 0.5 meters like a turtle, turn left by 30 degrees, and then turn right by 90 degrees.
```

The terminal prints the decision planning and execution process:




## Case 2: Anthropomorphic Dance Performance and Fun Joke Telling

After waking up yahboom, speak the instruction to the car:

```
Please start by performing a dance, and then tell me a joke about a little cat and a little dog.
```

The terminal prints the task response log:




**Execution Process and Phenomena**:

1. The large model parses out the `dance` action; the robotic arm servos sway and cooperate with slight chassis rotation, presenting a nimble dance movement;
2. After the action is complete, the large model improvises a lively and humorous joke and reads it aloud to the user in real time through speech synthesis.

---

# 4. Core Source Code Analysis

Source code package path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

## 4.1 Voice Acquisition and Multimodal Decision Flow (model_service.py)

In `model_service.py`, in voice mode, the user's natural language input is obtained through offline or online ASR recognition:

```
# Voice recognition callback obtains the text query
user_query = self.asr_process(audio_data)

# Feed the query into the large model to generate a standardized action queue
actions = self.generate_actions(user_query)
```

Example of the structured action list parsed by the large model:

```
#Move forward / backward
[
  {"action": "set_cmdvel", "params": {"linear_x": 0.3, "angular_z": 0.0, "time": 3.3}},
  {"action": "set_cmdvel", "params": {"linear_x": -0.08, "angular_z": 0.0, "time": 6.25}}
  {"action": "finish_dialogue", "params": {"text": "Report to master, I have completed all
actions as required!"}}
]

#Turning template
[
  {"action": "move_left", "params": {"angle": 90, "angular_speed": 0.5}},
  {"action": "move_right", "params": {"angle": 30, "angular_speed": 0.5}}
]
```

## 4.2 Low-Level Action Execution and Safety Limits (action_service_*.py)

After the service side receives the action queue, `CustomActionServer` schedules the low-level execution:

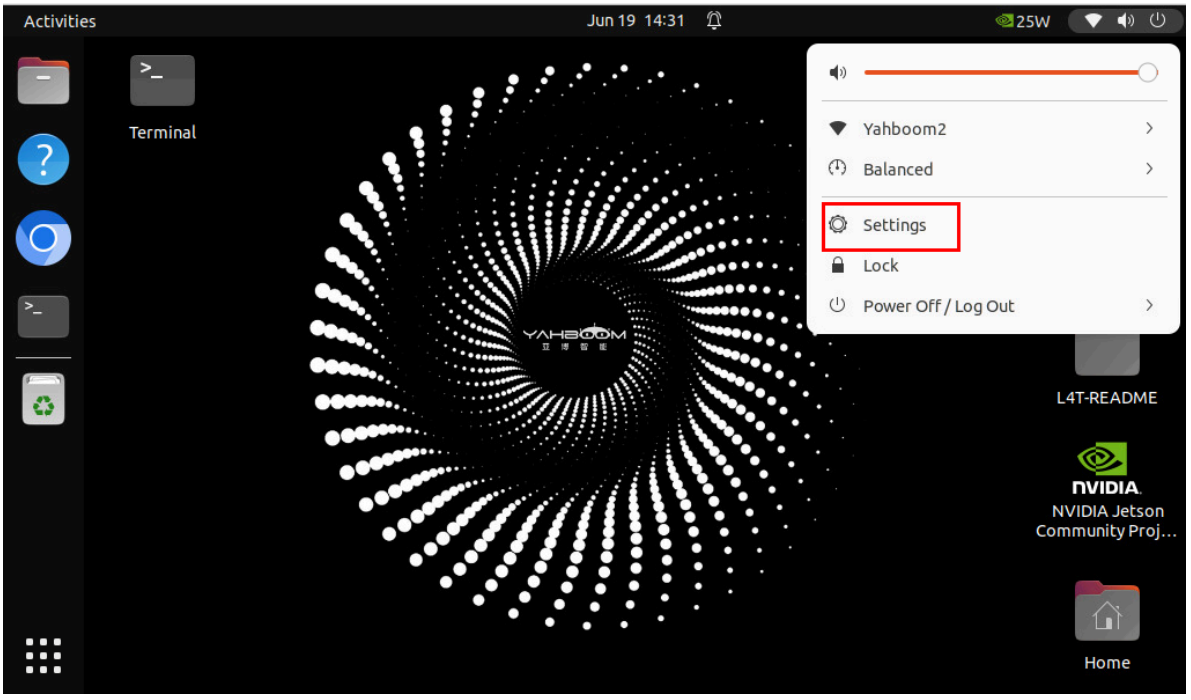
- `set_cmdvel`: Broadcasts velocity to `/cmd_vel` and precisely controls the travel distance according to time; after the task is complete, it forcibly publishes a zero velocity for safe braking;
- `dance`: Calls the servo angle array to control the robotic arm and present a coordinated dance pose;
- `finish_dialogue`: Calls the TTS voice module to read the reply text, bringing the task cycle to a complete closed loop.

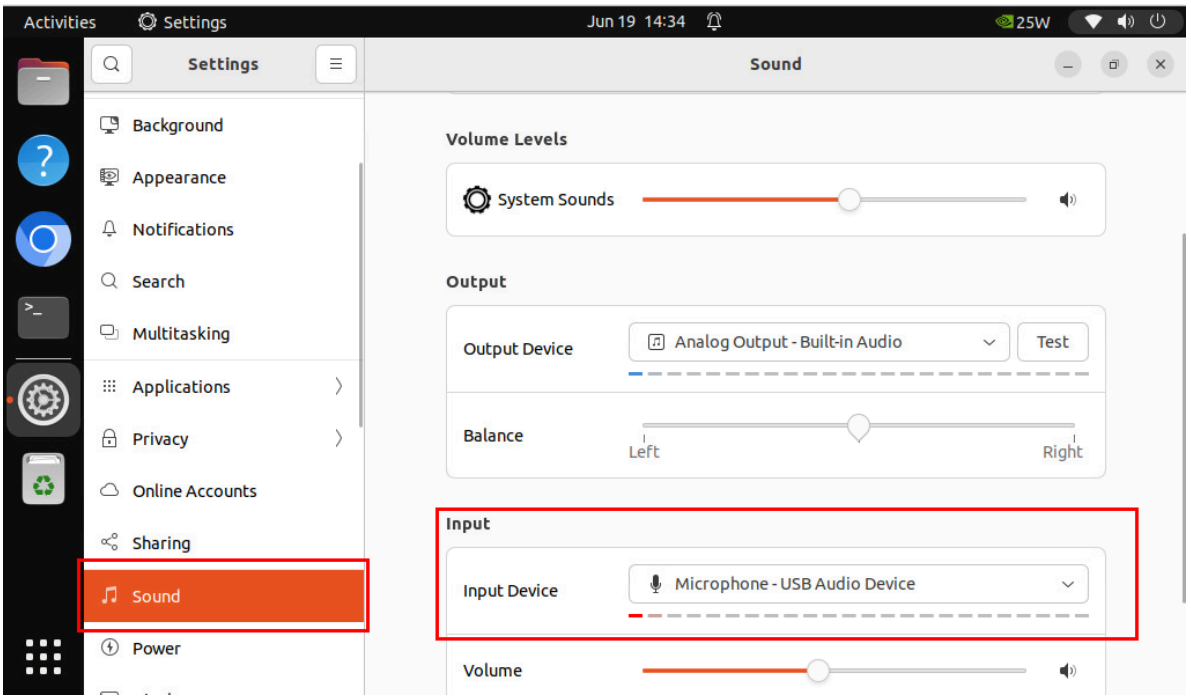
---

# 5. Common Troubleshooting and Audio Debugging

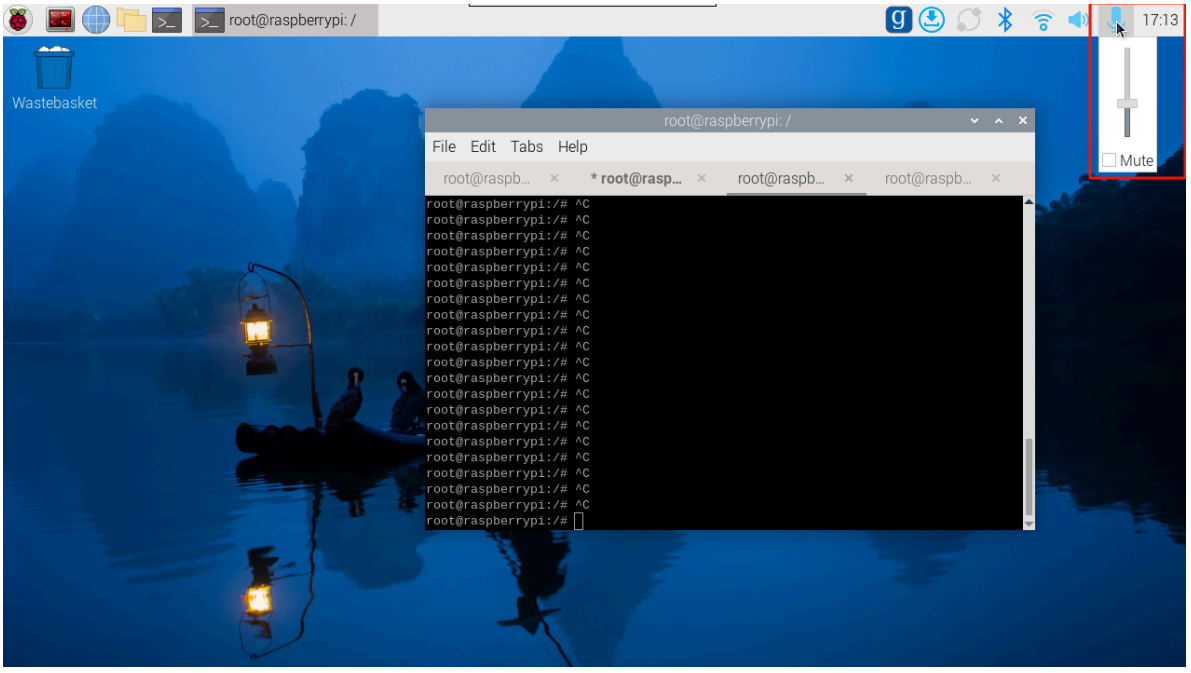
## 5.1 Microphone Recording Is Too Sensitive (VAD Cannot Truncate)

If VAD keeps showing `1` after you finish speaking and cannot stop recording, it means the ambient noise is too loud or the microphone gain is too high:

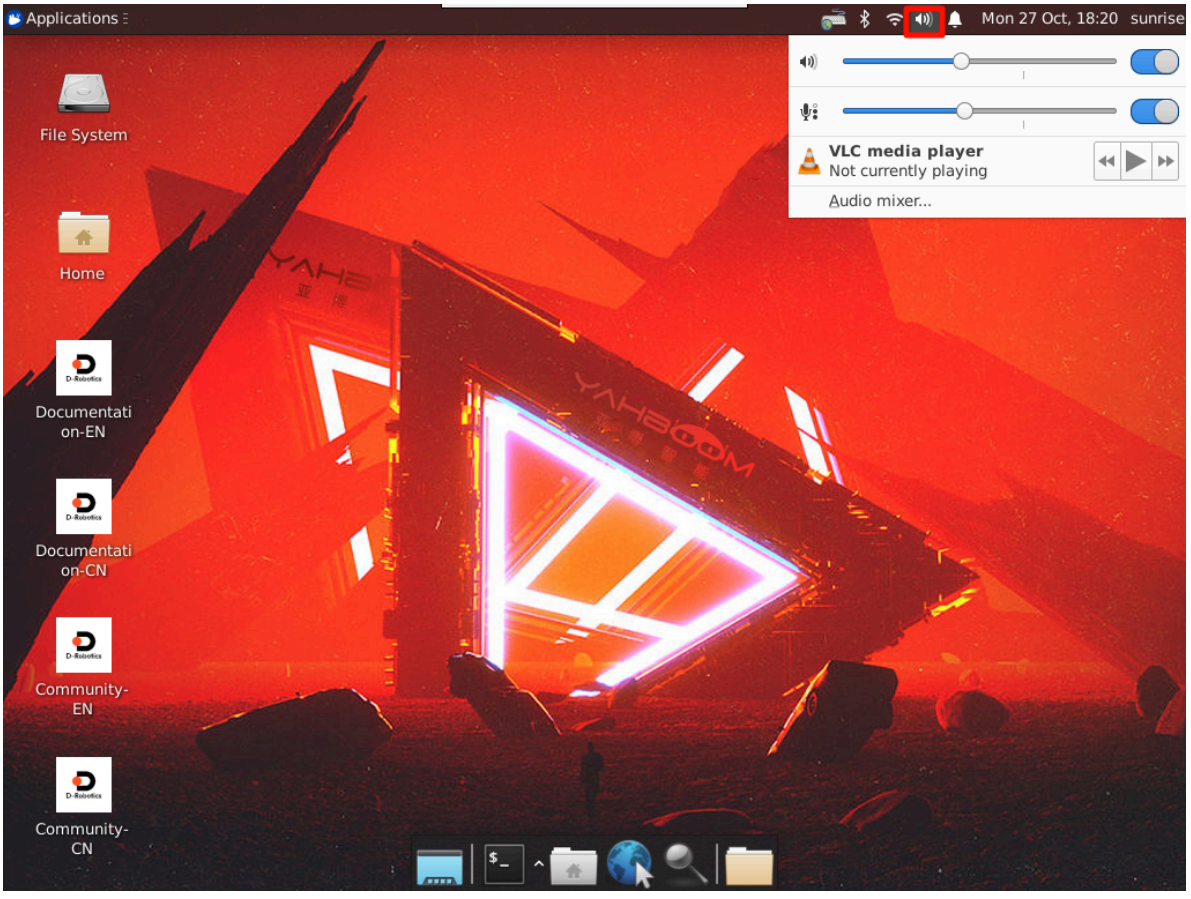
- **Orin host**: Connect to the desktop through VNC, open [Settings] $\rightarrow$ [Sound] $\rightarrow$ [Input], and lower the Volume slider appropriately:
  - 

  - 

- **Raspberry Pi 5**: Click the microphone icon in the upper-right corner to adjust the input gain:




- **RDK X5**: Click the speaker/audio settings in the upper-right corner to adjust the recording volume:




## 5.2 Microphone Recording Is Not Sensitive Enough (Truncated Prematurely)

If the speaker is too far from the car, VAD may fail to detect the faint sound. You can appropriately increase the microphone gain according to the steps above.

## 5.3 Poor Voice Hardware Contact

If a message indicates that the audio device is not found, unplug and replug the USB interface of the voice module or change the USB port, and try plugging/unplugging several times and restarting the program:

